

Practical Exploration of Decision Trees for Classification

Kristina Bhandari¹ and Namita Bhatta²

¹Department of Electronics and Computer Engineering, IOE, Thapathali Campus, Kathmandu 44600, Nepal

Corresponding author: Kristina Bhandari (e-mail: christinabhandari60@gmail.com).

ABSTRACT Principal Decision Trees are versatile Machine Learning algorithm that can perform both classification and regression tasks, and even multioutput tasks. They are very powerful algorithms, capable of fitting complex datasets. It has a hierarchical, tree structure, which consists of a root node, branches, internal nodes, and leaf nodes. The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. This study aims to classify plants in a nursery dataset into categories based on their attributes. It evaluates the effectiveness of classification algorithms using metrics like information gain, classification error, entropy, and Gini index.

INDEX TERMS Decision Tree, Supervised Machine Learning, Entropy, Gini Index.

I. INTRODUCTION

A decision tree is a non-parametric supervised learning algorithm, which is utilized for both classification and regression tasks. It has a hierarchical, tree structure, which consists of a root node, branches, internal nodes and leaf nodes. They are like step-by-step guides that help machines make decisions by breaking down complex problems into simpler choices. Every step in the tree denotes a query regarding the data, with varying possible answers. The decision tree divides the dataset into subsets recursively depending on the attribute value that yields the optimal partition based on a given criterion, like information gain or Gini impurity. The terminal nodes, or leaves, of the tree, reflect the ultimate choice or forecast at that moment. This process continues until the algorithm decides that future splits would not yield relevant information.

Decision trees are adaptable in a wide range of applications due in part to their ability to handle

both numerical and categorical input. Without a great deal of data preprocessing, they are able to represent intricate relationships and interactions between variables. Decision trees also have the advantage of being very interpretable; the logic of the model is simple to see and comprehend, which is especially helpful in areas where clear decision-making procedures are necessary. The development of decision trees may be traced back to the groundbreaking work of scientists like Ross Quinlan in the 1950s and 60s, who created algorithms like C4.5 and ID3 (Iterative Dichotomiser 3). These early algorithms introduced essential concepts like attribute selection measures and tree pruning, which are still foundational in modern decision tree techniques. The family of decision tree techniques has grown over time to encompass more sophisticated approaches like Gradient Boosting Machines and Random Forests.

Whereas Gradient Boosting Machines steadily construct trees to rectify the mistakes of earlier trees, producing extremely accurate predictive models, Random Forests construct many decision trees and combine their predictions to increase accuracy and robustness.

Decision trees are now widely used in many different industries, such as marketing, banking, and healthcare. They are a vital tool for data scientists and analysts looking for interpretable models that can help them make important decisions because of their versatility in handling different types of data and their strong performance in both classification and regression tasks. Whether used for financial risk assessment, medical condition diagnosis, or customer behaviour prediction, decision trees offer a strong and user-friendly method for data-driven decision-making.

II. RELATED WORK

Patrick M. Murphy et al. [1] investigates the question of whether smaller decision trees that follow Occam's Razor are more accurate than bigger ones. Smaller trees are not always the most accurate; experiments on both synthetic and real-world datasets demonstrate that slightly larger trees frequently perform better on unseen data. This contradicts the demand for simplicity by implying that larger trees can produce more accurate results for complicated concepts, even though smaller trees might be appropriate in the absence of domain expertise. The study comes to the conclusion that decision tree algorithms ought to reevaluate their bias towards simplicity and weigh it against the advantages of bigger trees.

Lior Rokach et al. [2] surveys decision tree classifiers, crucial in statistics, machine learning, and data mining. It presents a unified framework for constructing decision trees top-down, discussing splitting criteria like Information Gain, Gini Index, and Gain Ratio, and pruning methods to avoid overfitting. Key algorithms such as C4.5 and CART are highlighted, providing both theoretical insights and practical guidelines for effective implementation of decision tree classifiers.

Dr. Neeraj Bhagarva et al. [3] explores the utilisation and efficacy of the J48 algorithm, a well-liked technique for generating univariate decision trees for data mining applications. It examines the method of categorizing data, which separates it into categories based on specific attributes. The study highlights how successful the J48 algorithm is while employing the Weka software and working with large datasets. Additionally explored in the study is the concept of multivariate decision trees, which use many attributes at each node to improve classification accuracy. The work concludes with the reporting of experimental findings showing the J48 algorithm's effectiveness in a range of data mining tasks.

III. METHODOLOGY

A. THEORY

Decision trees work by recursively splitting the data based on attribute values. The root node is the feature with the maximum information gain, classification error, or Gini index; additional nodes are added in accordance with these parameters. This process keeps on until it reaches a leaf node, which stands for the goal classification that a decision boundary has identified. Trees of decisions are quite interpretable. They can readily comprehend and visualize the decision rules they acquire, which qualifies them for industries like healthcare and finance where explainability and transparency are essential. Clear decision boundaries for the nodes are established through data analysis and classification using pertinent attributes. To evaluate a classification model's performance, a classification report is used. It provides a detailed summary of metrics that assess the model's effectiveness in classifying different classes or categories, typically organized in a table with rows for each class and columns for various evaluation metrics.

B. MATHEMATICAL FORMULA

Dataset Split based on Information Gain

Entropy is a measure of uncertainty or randomness in a system. It is a measure of uncertainty or randomness in a system. In the

context of information theory and machine learning, entropy quantifies the amount of information or uncertainty involved in predicting the value of a random variable. It's commonly used to determine the best feature to split on when building decision trees. It can be calculated as follows:

$$H(s) = - \sum_{i=1}^c p_i \log_2(p_i)$$

Where, c represents the number of classes, and p_i is the probability of class i.

For each attribute, the weighted average entropy is calculated after the split as:

$$H(s) = \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} \cdot H(S_v)$$

where:

- $H(S, A)$ represents the weighted average entropy after the split using attribute A.
- v represents each unique value in the set of values of attribute A.
- $|S_v|$ denotes the number of instances in the dataset S that have the attribute value v.
- $|S|$ denotes the total number of instances in the dataset S.
- $H(S_v)$ denotes the entropy of the subset of instances in the dataset S that have the attribute value v.

Finally, the information gain from the split is determined

as:

$$IG(S, A) = H(S) - H(S, A)$$

1. Dataset split based on Gini Index

The Gini index, also known as Gini impurity, is a measure used to determine the purity of a node in a decision tree. It measures the frequency with which a randomly selected element would be misclassified if its label were assigned at

random based on the node's label distribution. The Gini index for a node is calculated as follows:

$$Gini(S) = 1 - \sum_{i=1}^n p_i^2$$

Where,

- $Gini(S)$ represents the Gini index of a node or dataset S.
- p_i is the proportion of samples in the node or dataset that belong to class i.
- The summation is taken over all n classes.

The formula for splitting based on Gini Index is:

$$Gini\ Split(S, A) = \sum_{v \in \text{values}(A)} \left(\frac{|S_v|}{|S|} \cdot Gini\ Impurity(S_v) \right)$$

Where:

- $Gini\ Split(S, A)$ represents the Gini index of the split of dataset S based on attribute A.
- v represents each unique value in the set of values of attribute A.
- $|S_v|$ denotes the number of instances in the dataset S that have the attribute value v.
- $|S|$ denotes the total number of instances in the dataset S.
- $Gini\ Impurity(S_v)$ represents the Gini impurity of the subset of instances in the dataset S that have the attribute value v.

2. Performance Evaluation

Following measures are used for evaluating model's performance:

Precision: Precision is a metric used in statistics and machine learning to evaluate the performance of a classification model, especially for binary classification tasks.

$$Precision = \frac{TP}{TP + FP}$$

Where,

- True Positives (TP) are the number of correctly predicted positive instances.

$$Accuracy: \frac{TP + TN}{TP + TN + FP + FN}$$

Where,

- TP (True Positives): These are the instances that are actually positive and were correctly predicted as positive by the model.
- TN (True Negatives): These are the instances that are actually negative and were correctly predicted as negative by the model.
- FP (False Positives): These are the instances that are actually negative but were incorrectly predicted as positive by the model.
- FN (False Negatives): These are the instances that are actually positive but were incorrectly predicted as negative by the model.

F1 Score: The F1 score is a metric used to evaluate the performance of a classification model, particularly when dealing with imbalanced classes. It combines precision and recall into a single metric and provides a balance between these two measures.

$$F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Where,

- False Positives (FP) are the number of incorrectly predicted positive instances (instances predicted as positive but are actually negative).

Accuracy: In machine learning and statistics, accuracy is a measure of a model's overall performance in producing accurate predictions. It calculates the percentage of all predictions produced by the model that come true. The formula for calculating accuracy is:

- Precision quantifies the model's accuracy in making positive predictions. This can be expressed as the ratio of true positives, or positively anticipated cases, to the total of false positives, or positively predicted instances that were mistakenly forecasted.
- Recall, which is often referred to as sensitivity or true positive rate, quantifies the model's accuracy in identifying positive cases. It is the proportion of real positives to the total of both true positives and false negatives, or cases that were misclassified as negative.

C. SYSTEM BLOCK DIAGRAM

Figure 1 illustrates the process of constructing a decision tree for classification tasks. It begins with the dataset, which is the input for the entire process. The dataset is first split into training and validation sets to ensure that the model is evaluated properly. Two primary calculations are performed on the training set: Entropy Calculation and Gini Index Calculation. The entropy calculation leads to information gain, determining the best features for splitting the data. Simultaneously, the Gini index helps identify the feature that creates the purest subsets. The feature with the lowest Gini index is selected for splitting. Based on these calculations, leaf nodes are created, representing endpoints where decisions are made. These leaf nodes are assigned class labels based on the majority class of instances reaching them. Finally, the decision tree is constructed using these nodes and labels, resulting in a model capable of classifying new

data instances accurately by following learned splits and decisions. This process ensures an optimal decision tree through thorough impurity assessments and strategic feature selection.

D. INSTRUMENTATION TOOLS

The entire process was carried out using Python within the Google Colaboratory platform, which is a convenient online tool for coding and running data analysis tasks. We used several Python library to tackle the problem.began by loading our dataset and using Pandas to visualize and understand its structure. The NumPy library was employed for efficient numerical operations and array manipulation, allowing for seamless handling of numerical data. This step helped us grasp the initial insights and prepare the data for further analysis. Next, we utilized functions from the scikit-learn library to construct a Decision Tree Classifier. This classifier is a powerful tool in machine learning that learns from the data to make predictions or decisions. Visualizing the results of this model was crucial, so we employed different visualization tools like Seaborn and matplotlib to create clear graphs and charts. These visual representations allowed us to interpret how well our model was performing and identify any meaningful patterns or trends in the data. Finally, we evaluated the model's performance using the confusion matrix, accuracy, precision, recall, and F1 score.

E. WORKING PRINCIPLE

1. Dataset Preparation

The 'Nursery' dataset was taken from the UC Irvine Machine Learning Repository. The dataset is loaded from the scikit learn library. The feature names, the target classes and the attributes are then loaded to a Pandas dataframe and the values are visualized. The dataset is then split into training and validation dataset in the ratio of 67:33.

2. Decision Tree Classifier Training

A decision tree classifier is initialized using the scikit-learn library, with dataset splitting performed using both the Gini index and entropy for information gain. Next, the training dataset is used to train the classifier. The validation set's values are then predicted using the learned classifier. The actual labels of the validation

dataset are compared with the projected outcomes.

3. Data Analysis and Visualization

After training the decision tree classifier using both the Gini index and entropy for dataset splitting, the model's structure is visualized using libraries like Matplotlib. A confusion matrix is then generated to compare its predictions with the actual values from the validation dataset, providing insights into its performance

IV. RESULTS

A. DECISION TREE USING ENTROPY

Using the Nursery dataset, we worked on constructing decision trees for classifying between target classes of the dataset taking the first 2000 data from the dataset.

Firstly, we built a decision tree using the information gain from entropy dropping some unimportant features. We built a tree with its maximum depth. The decision tree shown in Figure 2 Decision Tree using Information Gain shows the tree with its maximum depth without limitations on the depth i.e. without pruning. The depth of the tree is 8. From the figure, we can conclude that the maximum depth of the tree is 8. The green color represents priority class, orange class represents not recommended. Likewise, the purple color represents the 'very recommended' class.

Confusion Matrix:

From Figure 3, we can see that high density of data falls under the principal diagonal. This indicates that the number of validation dataset is correctly classified. However, there are few cases of very recommend being labeled as priority and priority being labeled as very recommended.

Classification Report:

Table 1, shows the classification report for entropy-based classifier. It summarizes the performance of a multiclass model with an

overall accuracy of 95%. It excels in predicting classes 'not_recom' and priority with high precision (1.00 and 0.94 respectively) and recall (1.00 and 0.97 respectively), indicating robust identification of these classes. However, for 'Very_recom', precision drops to 0.77 and recall to 0.63, suggesting some difficulty in accurately predicting this class. Overall, the model achieves a balanced F1-score of 0.94, reflecting strong performance across most classes, with 'Very_recom' showing potential for improvement in predictive accuracy.

Similarly, we rebuilt decision tree with reducing its depth i.e pruning. The decision tree shown in Figure shows the tree with depth=4. The green color represents 'priority' class, orange class represents 'not recommended'. Likewise, the purple color represents the 'very recommended' class.

After limiting depth of tree to 3, we can see following changes:

Decision Tree at depth 3:

Figure 4 Decision Tree at depth 3 using Entropy-based Classifier, illustrates decision tree using entropy-based classifier at depth 3. It has only two classes. The green color in tree represents 'priority' class, orange class represents 'not recommended'.

Confusion matrix:

Figure 5 Confusion Matrix at depth 3 using Entropy-based Classifier, illustrates confusion matrix after its depth is limited to 3. However, there are no predictions for 'very_recom' correctly classified. Thus, all instances of 'very_recom' were misclassified

Classification report:

The classification report at Table 2 shows that performance is degraded after pruning. Precision

and recall for classes like 'very_recom' has been reduced to 0.00. The overall accuracy is also reduced to 90%.

B. DECISION TREE USING GINI INDEX

Gini Index has similar process for building decision trees as that of Entropy. Also, decision trees constructed using the Gini Index have similar structures to those built using Entropy. However, the main difference lies in the splitting criteria, which is determined by the Gini Index.

Figure 6 shows decision tree using GINI index has similar depth as that of the tree constructed using entropy-based classifier.

Confusion matrix:

Figure 7, shows the confusion matrix using GINI index. It shows that higher number of 'very_recom' classes labelled correctly than that of them matrix of entropy-based classifier.

Classification Report:

The classification report in Table 3, using the GINI index shows a similar report as that of the entropy-based classifier.

After limiting depth of tree to 4, we can see the following changes:

Decision Tree at depth 4:

Figure 8, illustrates decision tree using entropy-based classifier at depth 3. In this case, with a depth of 3, the tree consists of three levels of nodes. The nodes are color-coded for clarity: green nodes typically represent nodes where the majority class is 'priority', while orange nodes indicate 'not recommended' class.

Confusion matrix:

Figure 9 illustrates confusion matrix after its depth is limited to 3. It depicts less number of 'very_recom' being correctly predicted.

Classification report:

The classification report at Table 4, illustrates performance after pruning which shows similar results as that of tree without limited depths with accuracy 95%

V. DISCUSSION AND ANALYSIS

The previous sections demonstrated experimental results obtained using the Decision Tree Classifier using the Nursery dataset. It helped us grasp valuable insights of decision trees on its performance and utility in classification tasks. By dividing the dataset into subsets according to feature values, the decision tree method produced a hierarchical structure that made it easier to classify instances into predetermined groups like "priority," "not recommended," and "very recommended." We could see how the model generated decisions based on the most discriminative attributes, represented by nodes in the tree, by visually examining the decision tree at different depths.

Firstly, we divided the dataset into training and testing sets, with a 70% training and 30% testing split to ensure effective results. We then employed both the Gini index and information gain (entropy) to construct decision trees. Initially, we built a decision tree using entropy without any depth limitation. This tree, at its maximum depth, achieved an accuracy of 95%. However, it was also quite complex. To address this, we pruned the tree by reducing its depth to three. This simplified tree had fewer nodes, making it less complex. However, the accuracy of the pruned tree decreased to 90%. Also, the confusion matrix after pruning showed that "very_recom" has less number of correct predictions. Pruning the tree affected its performance by reducing its complexity and the number of nodes, which can help prevent overfitting and improve generalization. However, this simplification came at the cost of a slight decrease in accuracy, demonstrating the trade-off between model complexity and performance.

After that, we employed GINI index once the dataset was split. As similar to using entropy, we initiate constructing tree at maximum depth. The tree delivered an outstanding performance as similar to that of using entropy with 95% accuracy. It was found to have 8 depths and was a little complex to visualize. After pruning the depth to 4, the confusion matrix showed 'not_recom' class to have a less correct prediction but it delivered around similar accuracy even after pruning.

Regardless of the splitting criterion entropy or the Gini INDEX, the decision tree performed consistently well. The results obtained from both methods were similar in terms of F1 score, accuracy, precision, and recall. In terms of efficiency, for node splitting in decision trees, both entropy and Gini are computationally efficient. But compared to entropy, the Gini index may be computed a little bit faster because it just needs to calculate squared probabilities, whereas entropy requires logarithmic calculations. This slight variation in computing time is negligible unless dealing with very huge datasets or deep trees. The decision tree maintained similar performance metrics using either criterion, therefore depending on the particulars of the dataset and interpretability preferences, either Gini or entropy would be appropriate options.

VI. CONCLUSION

Analyzing the Nursery dataset using a Decision Tree provided valuable insights. This lab followed the principles of employing a Decision Tree Classifier, demonstrating its effectiveness in this specific context. The primary objective of classification was successfully achieved by accurately representing the decision rules derived from the Nursery dataset. The Decision Tree Classifier successfully captured the patterns and relationships within the Nursery dataset by taking into account a variety of parameters and choosing the right decision rules, which produced accurate classification results. The investigation shows that a number of

parameters, such as the depth of the tree and the features chosen that are relevant to the breast cancer dataset, affect the Decision Tree Classifier's performance.

In general, deeper trees are more accurate in identifying cases of Nursery; nevertheless, it is important to balance the depth of the tree to prevent overfitting while retaining pertinent information. The decision rules play a crucial role in the feature selection process, which determines how well Nursery class labels are predicted. The results demonstrate the Decision Tree Classifier's effectiveness in handling the Nursery dataset and providing interpretable models. The process of identifying the most influential characteristics is aided by the

classifier's capacity to facilitate feature importance analysis. The Nursery dataset provides evidence of the Decision Tree Classifier's effectiveness in classifying cases. Reliable and comprehensible categorization outcomes are attained through meticulous consideration of depth, feature selection, and interpretation of decision guidelines. This approach uncovers underlying patterns and relationships within the Nursery dataset, making it a powerful tool for classification tasks.

Although Decision Tree Classifier is an effective and powerful tool, it has some limitations. A significant drawback is that when the tree is allowed to grow up to certain depth, it overfits the training data.

REFERENCE

- [1] P. M. Murphy and M. J. Pazzani, "Exploring the Decision Forest: An Empirical Investigation of Occam's Razor in Decision Tree Induction," 1994.
- [2] L. Rokach and O. Maimon, "Data mining with decision trees. Theory and applications," 2008.
- [3] D. N. Bhargava, G. Sharma, D. R. Bhargava and M. Mathuria, "Decision Tree Analysis on J48 Algorithm for Data Mining," 2013.



KRISTINA BHANDARI, a fourth year computer engineering undergraduate student at IOE, Thapathali Campus. She is eager to explore the latest advancements in AI and aims to make significant contributions to the field. (THA077BCT022)



NAMITA BHATTA, a fourth-year computer engineering undergraduate at IOE, Thapathali Campus. Her goal is to leverage her knowledge and expertise in

AI to make a significant impact in the field.
(THA077BCT048)

APPENDIX

I. FIGURES

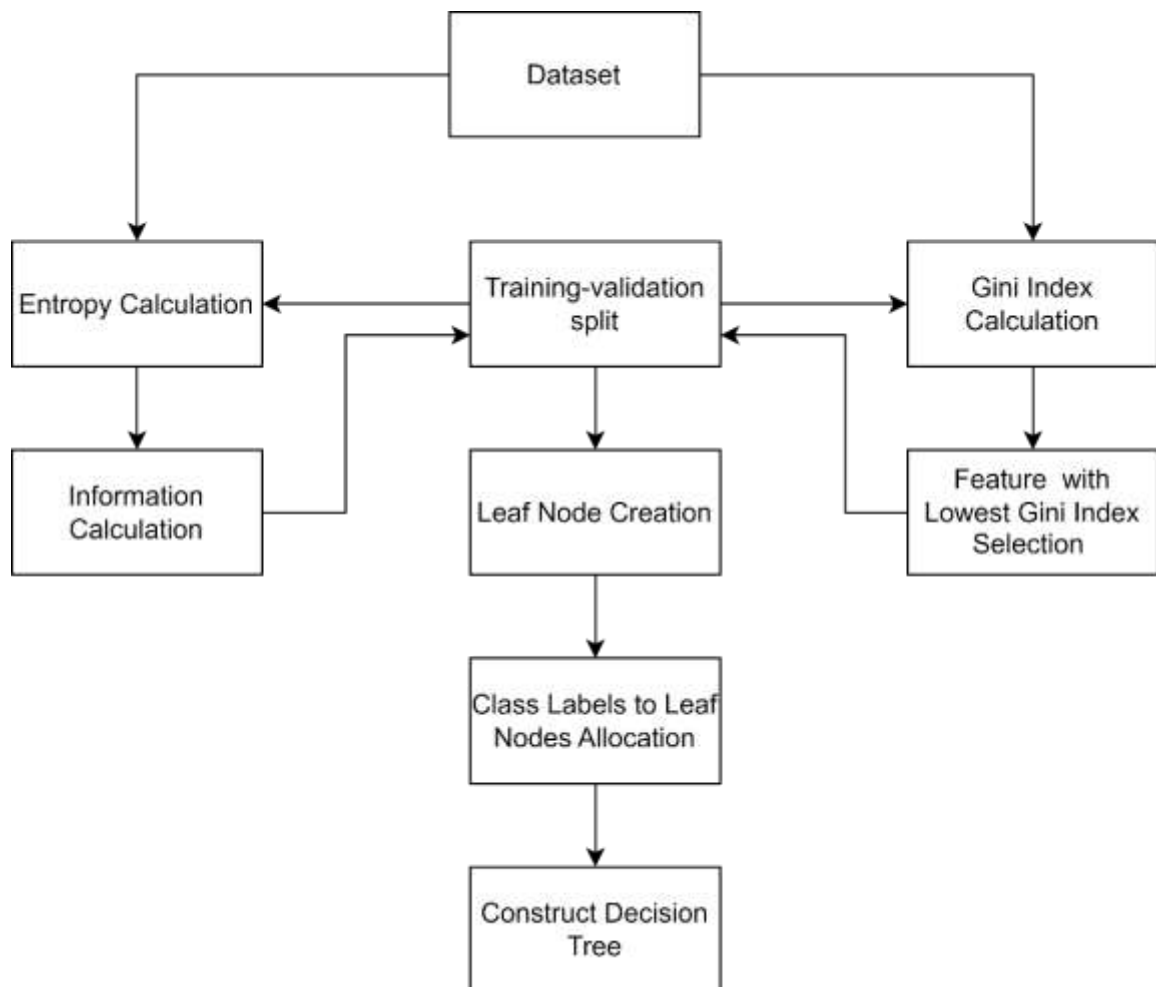


Figure 1 System Block Diagram of Decision Tree

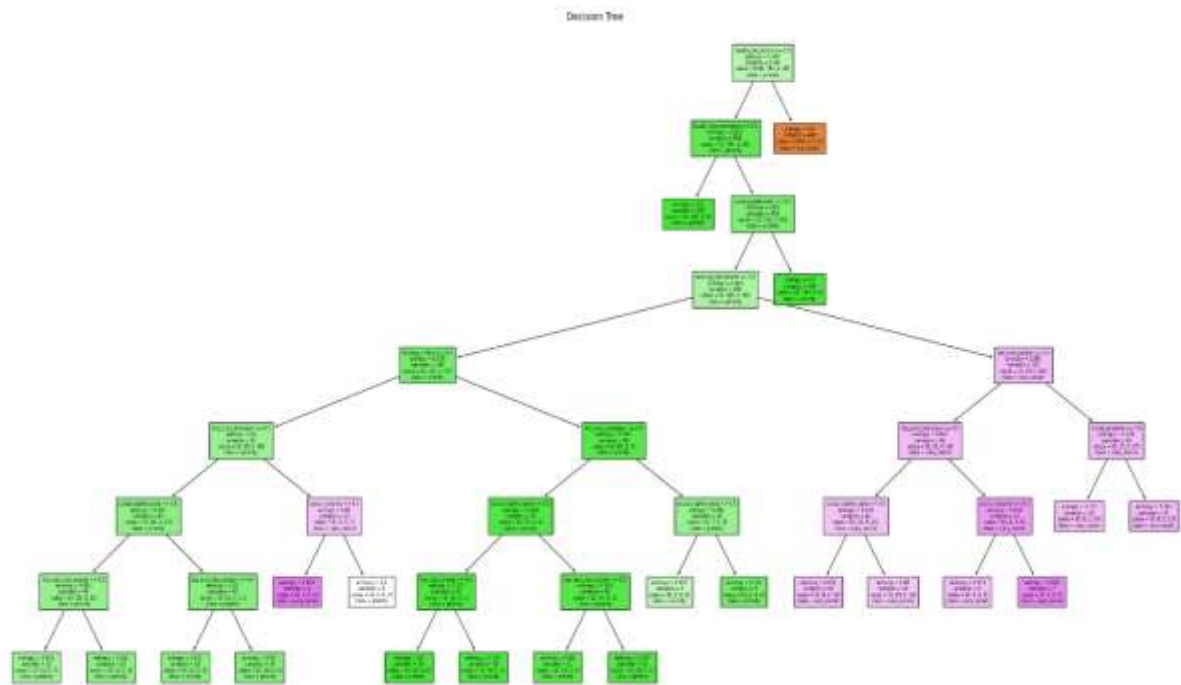


Figure 2 Decision Tree using Information Gain

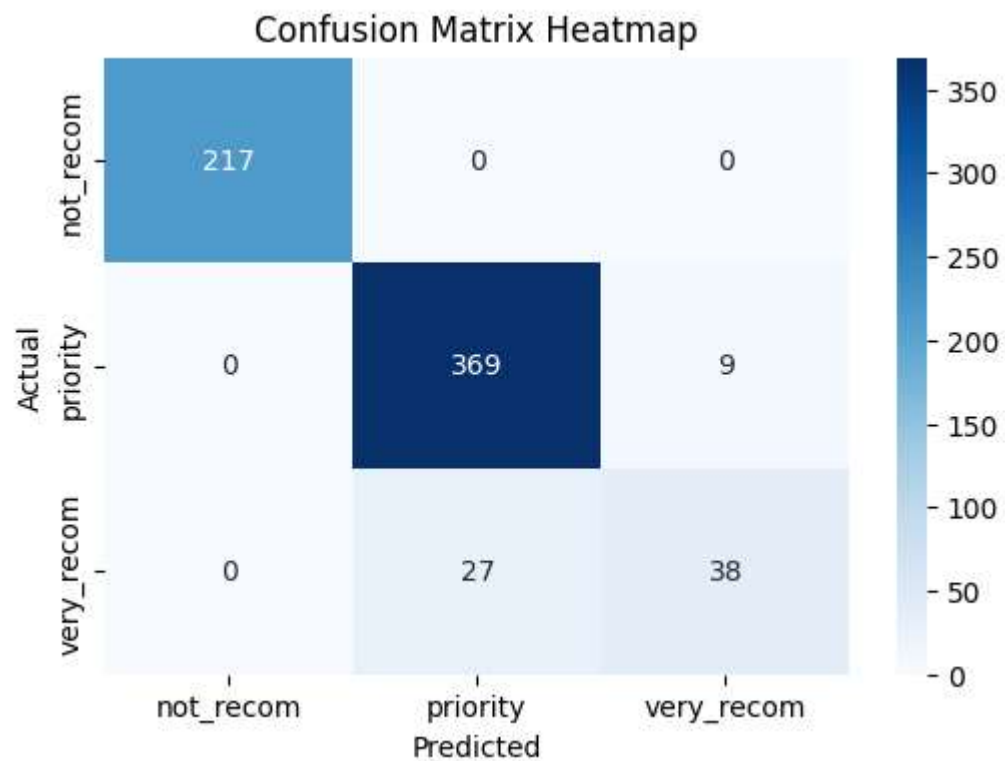


Figure 3 Confusion Matrix for Entropy-based Classifier

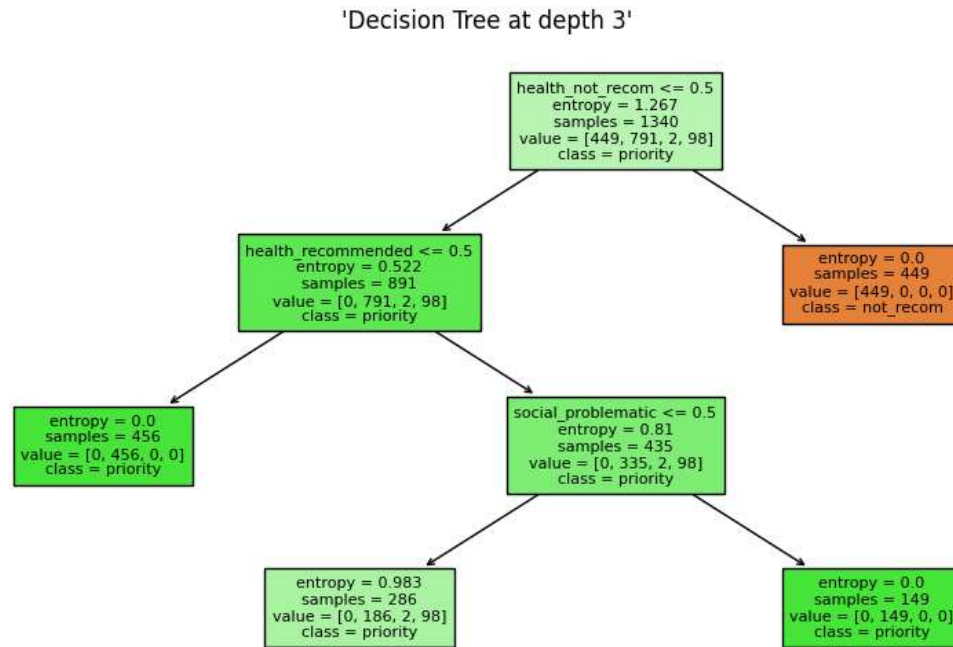


Figure 4 Decision Tree at depth 3 using Entropy-based Classifier

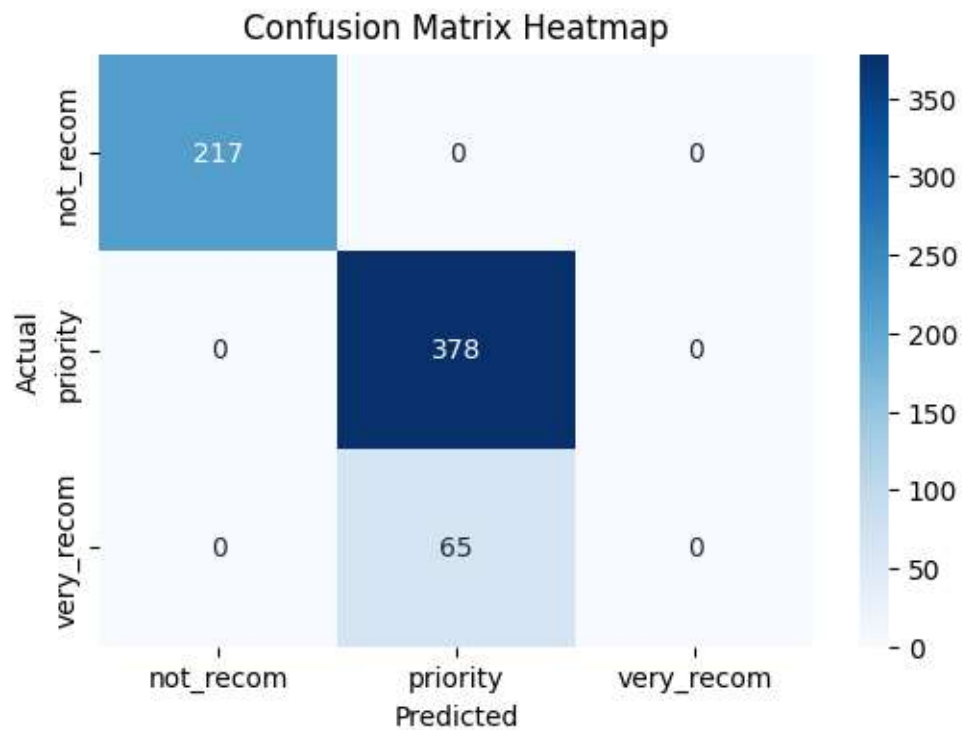


Figure 5 Confusion Matrix at depth 3 using Entropy-based Classifier

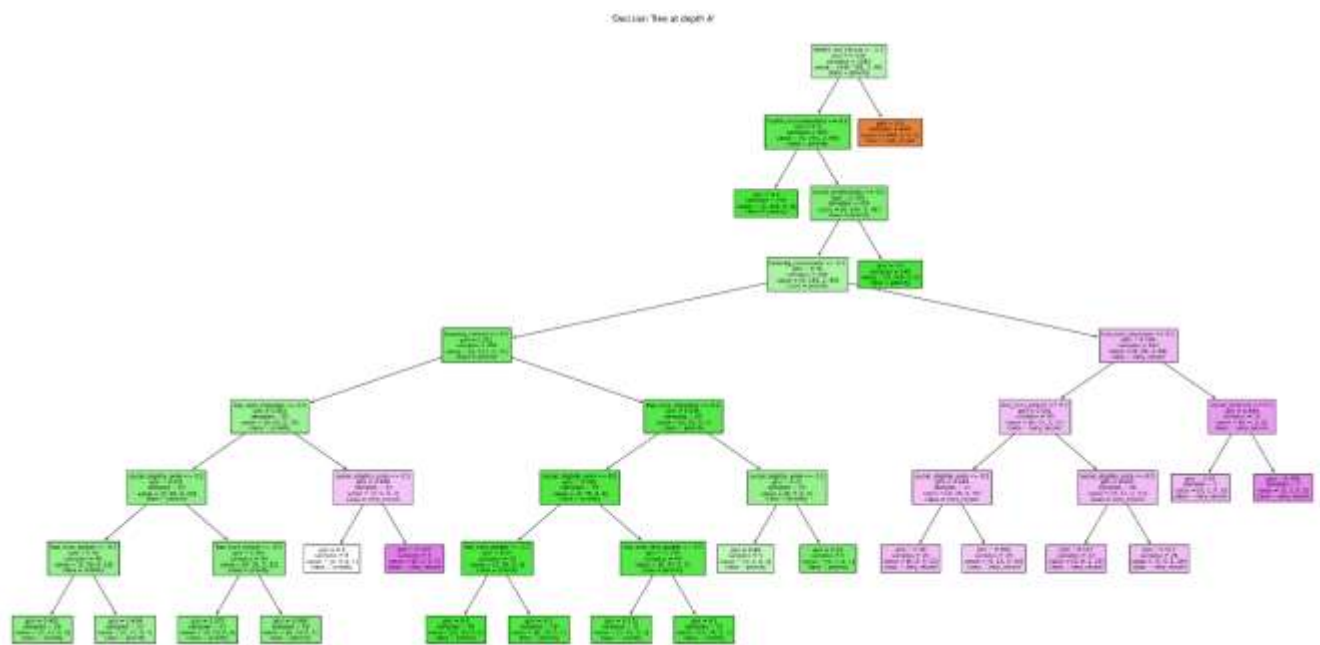


Figure 6 Decision Tree using GINI Index

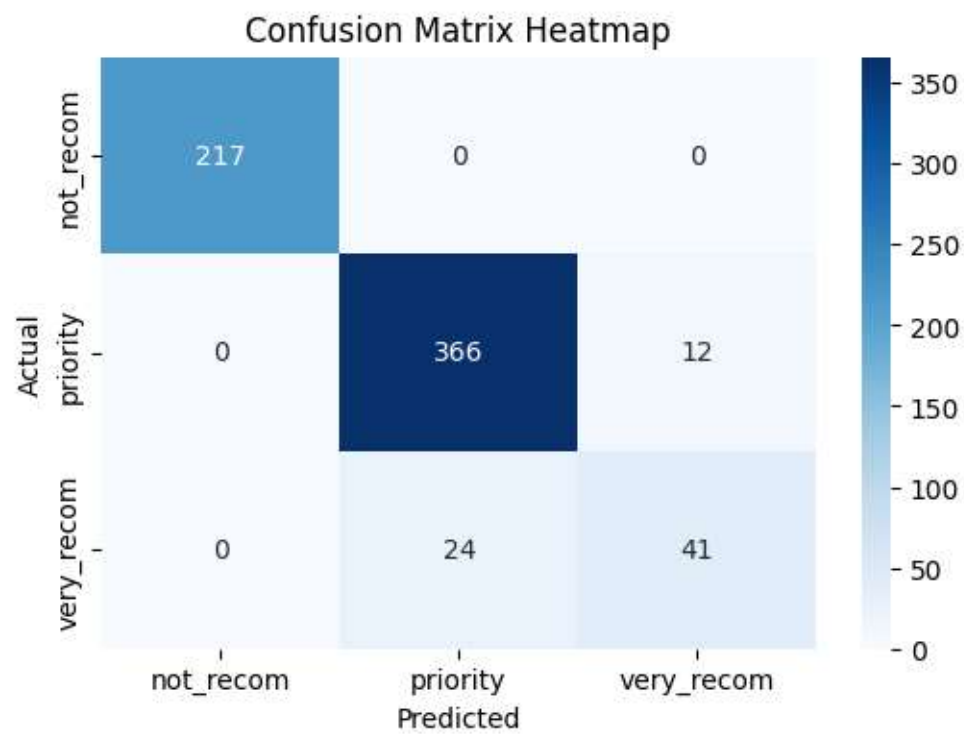


Figure 7 Confusion Matrix using GINI Index

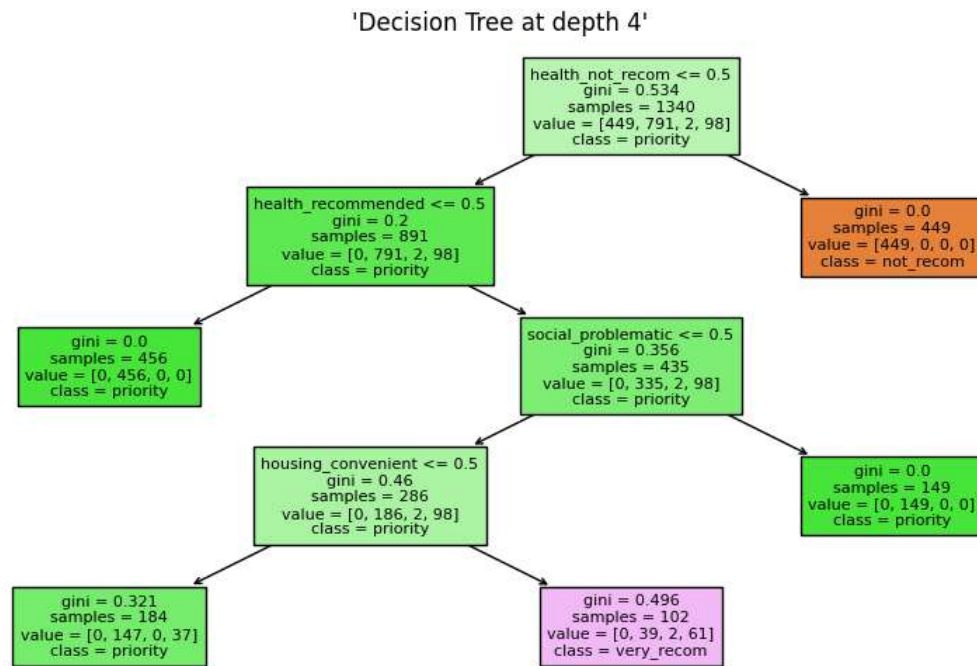


Figure 8 Decision Tree at depth 4 using GINI Index

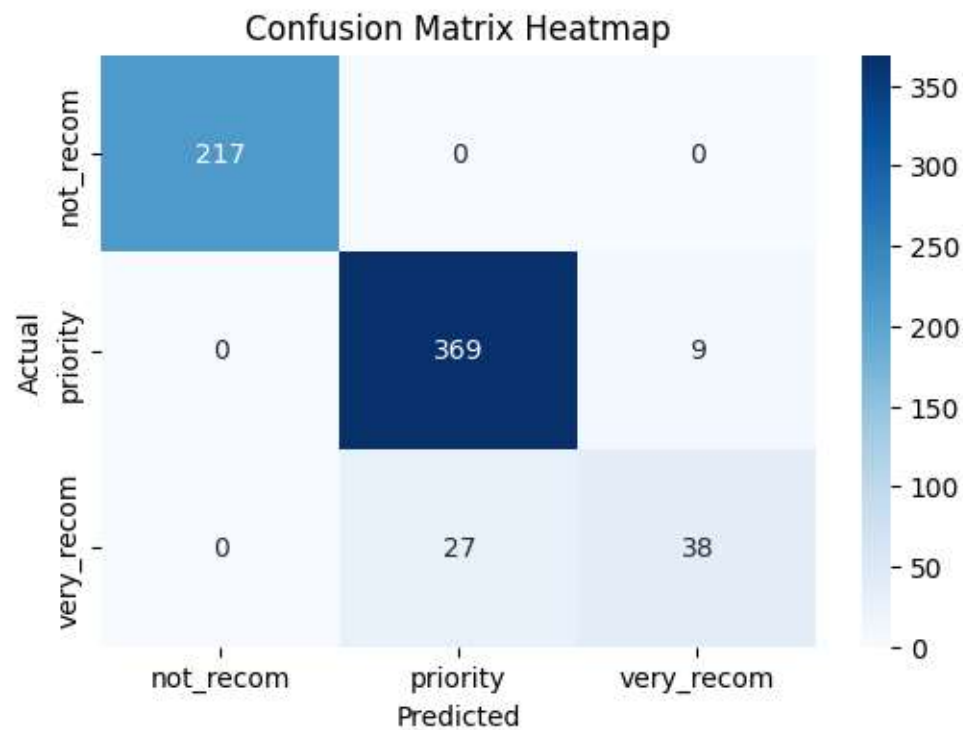


Figure 9 Confusion Matrix at depth 4 using GINI Index

II. Tables

Table 1 Classification Report using Entropy-based Classifier

Class	precision	recall	f1-score	support
not_recom	1.00	1.00	1.00	217
priority	0.94	0.97	0.95	378
Very_recom	0.77	0.63	0.69	65
accuracy			0.95	660
Macro avg	0.90	0.87	0.88	660
Weighted avg	0.94	0.95	0.94	660

Table 2 Classification Report using Entropy-based Classifier at depth=3

Class	precision	recall	f1-score	support
not_recom	1.00	1.00	1.00	217
priority	0.85	1.00	0.92	378
Very_recom	0.00	0.00	0.00	65
accuracy			0.90	660
Macro avg	0.62	0.67	0.64	660
Weighted avg	0.82	0.90	0.86	660

Table 3 Classification Report Using Gini Index

Class	precision	recall	f1-score	support
not_recom	1.00	1.00	1.00	217
priority	0.94	0.97	0.95	378
Very_recom	0.77	0.63	0.69	65
accuracy			0.95	660
Macro avg	0.90	0.87	0.88	660
Weighted avg	0.94	0.95	0.94	660

Table 4 Classification Report using GINI index at depth=5

Class	precision	recall	f1-score	support
not_recom	1.00	1.00	1.00	217
priority	0.93	0.98	0.95	378
Very_recom	0.81	0.58	0.68	65
accuracy			0.95	660
Macro avg	0.91	0.85	0.88	660
Weighted avg	0.94	0.95	0.94	660

